

CPU Algorithm Design

Exercise 3

- Submit electronically (MOODLE) as one archive named after the group members.
- Archive should encompass a PDF and your code.
- Include names on the top of the sheets.
- A maximum of 3 students is allowed to work jointly on the exercises.
- Points is capped at 40

File List

1. `plotRes.py`: plotting script
2. `reduce_Unroll_main.cpp` and `transform_Unroll_main.cpp`: example of how to run the code with small N for debug purpose.
3. `reduce_loopUnrolling.hpp` and `transform_loopUnrolling.hpp`: reduce and transform using normal containers. You shouldn't change these files.
4. `reduce_loopUnrolling_view.hpp` and `transform_loopUnrolling_view.hpp`: reduce and transform using `views(std::array)`. You should complete these files.
5. `reduce_Unroll_bench.cpp` and `transform_Unroll_bench.cpp`: benchmarking code for reduce and transform using containers. You should submit the executable built from these to the server.
6. `reduceV_Unroll_bench.cpp` and `transformV_Unroll_bench.cpp`: benchmarking code for reduce and transform using `views(std::array)`. You should submit the executable built from these to the server.
7. `load_env_CPUAD.sh`: script to load the dependencies to your session. You shouldn't change this.
8. `ex03SlurmScript.sh`: script to submit your executable to the server. Base on your need, you should change the `rome00`, the name of the executable and the name of the output file.
9. `utilities.hpp` and `benchmark.hpp`: utility functions for the code and benchmarking. You shouldn't change these.
10. `CMakeLists.txt`: file for cmake to generate the makefile. You shouldn't change these.(for debug purpose, you can change the `-O3` to `-g` at 10th line, but make sure the result is generated with `-O3` flag)
11. `UNROLLFACTOR.h` `benchUnrollFactor.cpp` and `unrollScript.sh` utilities files for the second problem.

1 Adapting reduce and transform (20 points)

1. Complete the `reduce_loopUnrolling_view.hpp` except for two xsimd function and `benchReduceUnrollTreeDirective`. Like `ex02`, in `reduce_loopUnrolling.hpp`, the data is loaded from the container which is in memory. However, the containers are initialized with uniform values. Now, in `reduce_loopUnrolling_view.hpp`, replace the container with `std::views::repeat`.
2. Complete the `transform_loopUnrolling_view.hpp` except for the last function (`benchTransformUnrollLoopPeelingDirective`). In `transform_loopUnrolling.hpp`, the data is loaded from the container which is in memory. However, the containers are initialized with values with specific pattern($a, a+1, a+2 \dots a+n$). Now, in `SIMD_transform_view.hpp`, replace the container with `std::ranges::views::iota`. For transform you also need to store the result into an container. As we're computing $W = a*V + W$, use `views::iota` for V , and a container(`std::vector`) of size 256 for W . For W , use modulo indexing ($i\%n$) for reading and writing. You could wrap your container with a `view` for those functions that doesn't access element with index. Or have an extra counter and perform iterator arithmetic.

Remark

When adapting the functions, please try to change as less code as possible. All the function inside of the same class perform the same task, copying the same code ten times will satisfy the functionality requirements but it's not what we are looking for. For example, if the original code uses `transform` with `std::execution::unseq`, then the adapted version should also uses `transform` with `std::execution::unseq`. Also make sure that the data type is still `Real`.

2 Adapting benchTransformUnrollLoopPeelingDirective. (15 points)

Complete the `benchTransformUnrollLoopPeelingDirective` in `transform_loopUnrolling_view.hpp`. First, also change the container type just like the other function in `transform_loopUnrolling_view.hpp`. Now use XSIMD with unaligned loads in the inner loop, for this insert a `static_assert` which ensures that `unroll_factor` is divisible by `batch::size`. How does performance change with `unroll_factor`? use the script file provided to run the function with different `unroll_factor`. To run the script, go to your build directory and type

```
../unrollScript.sh
```

The executables can be found in `changingUnrollFactor` directories and the results are in `res.`(both in your build directory)

Remark

Since now we're not using `std::array` as containers like in ex02, you don't need to `load_unaligned` the `V` anymore, use more efficient way to construct the `xsimd::batch<Real>`. You can assume `simd_width = 8`.

3 Adapting `benchReduceUnrollTreeDirective`. (15 points)

Change `benchReduceUnrollTreeDirective` to use a settable `constexpr auto tree_deg` (up to now we were using `tree_deg=4`, degree of the tree is the number of child a node has, for example, binary search tree have degree 2), i.e. we want code that will work for any setting of `unroll_factor` and `tree_deg` as long as `unroll_factor` is divisible by `tree_deg` (`static_assert` check). The tree needs to be as tall as possible. For example, if my `unroll_factor` is 96 and my `tree_deg` is 4, then here's the breakdown steps. 1. perform a reduce, now you have a problem of size 24. 2. perform another reduce, now you have a problem of size 6. 3. Since you problem size is no longer divisible by `tree_deg` anymore, you perform a naive sequential loop to add all 6 items together. If `unroll_factor` is 64 and `tree_deg` is 4, then this function should perform exact the same computation as `benchReduceUnrollTree`.

Hint

You can start by assuming `unroll_factor` is a power of `tree_deg`.

4 Adapting `benchReduceUnrollSimdXHorizontal` and `benchReduceUnrollSimdXVertical`. (15 points)

Drop the assumptions (`unroll_factor=64`, `simd_width=8`) in these functions, the code should work for different `simd_widths` as long as `unroll_factor` is divisible by `simd_width=batch::size` (`static_assert` check). Test by changing `Real` to `double`, because this changes `batch::size`. The final computation graph should also look like a multi-level tree just like the previous problem. For the horizontal version, your tree degree is `simd_width`. for vertical version, your tree degree is 2. Don't forget to change the container type. For reduction, just have a `std::array` of size `simd_width` to replace the container.

Hint

You can start by assuming `unroll_factor` is a power of `simd_width`(or 2).

5 Benchmarking (15 points)

Benchmark and analysis the performance just like in ex02 for all the functions. Now, what trends or patterns do you observe in the performance data?

Submission

Submit `reduce_loopUnrolling_view.hpp` and `transform_loopUnrolling_view.hpp`, the plots and the text answers.